GII-ADE-M. Relación de problemas. Tema 2-2. 22/10/2026

Indicaciones. Salvo que se indique lo contrario, se supone que las lecturas y escrituras de variables compartidas son atómicas, que las instrucciones de cada proceso se ejecutan en el orden escrito y que el planificador puede intercalar arbitrariamente los pasos de los procesos. Se supondrá planificación débilmente justa únicamente cuando el enunciado lo requiera explícitamente. Justificar todas las respuestas; un contraejemplo debe incluir una secuencia de ejecución suficientemente detallada.

54. Propiedades de corrección. Considerar las cuatro exigencias habituales del problema de la exclusión mutua: exclusión mutua, progreso o ausencia de interbloqueo, ausencia de inanición y espera acotada (equidad). Clasificar cada una como propiedad de seguridad o de vivacidad, precisando las hipótesis necesarias sobre la planificación. Explicar mediante ejemplos por qué garantizar exclusión mutua y ausencia de interbloqueo no implica necesariamente ausencia de inanición. ¿Qué se puede afirmar de un algoritmo que únicamente satisface las cuatro condiciones de Dijkstra estudiadas en clase? Explicitar qué formulación de dichas condiciones se utiliza.

55. Dos indicadores sin variable de turno. Sean dos procesos con las variables compartidas `b[0]` y `b[1]`, inicialmente falsas, y el siguiente protocolo, ejecutado por `P(i)` (`j=1-i`):

```
b[i] := true;
while b[j] do
    b[i] := false;
    while b[j] do skip;
    b[i] := true;
endwhile;
<seccion critica>;
b[i] := false;
```

Analizar por separado: (a) exclusión mutua; (b) ausencia de interbloqueo; (c) ausencia de inanición. Si alguna propiedad no se satisface, construir un escenario concreto. Comparar el papel de la variable de turno en este algoritmo y en el de Peterson para dos procesos.

56. El algoritmo de Hyman. Se comparte `c[0]=c[1]=1` y `turno=1`. Cada proceso `P(i)`, con `j=1-i`, ejecuta:

```
c[i] := 0;
while turno != i do
    while c[j] = 0 do skip;
    turno := i;
endwhile;
<seccion critica>;
c[i] := 1;
```

Determinar si el algoritmo garantiza exclusión mutua. Si no la garantiza, proporcionar una intercalación de instrucciones que lleve simultáneamente a ambos procesos a sus secciones críticas. Identificar exactamente la decisión de diseño que permite el fallo.

57. Peterson para dos procesos. Sean `quiere[0]=quiere[1]=false` y `turno` compartido. Cada `P(i)` ejecuta:

```
quiere[i] := true;
turno := 1-i;
while quiere[1-i] and turno = 1-i do skip;
<seccion critica>;
quiere[i] := false;
```

(a) Demostrar la exclusión mutua. (b) Argumentar el progreso y la ausencia de inanición, indicando las hipótesis de planificación. (c) Investigar qué sucede si se intercambia el orden de las dos primeras asignaciones. (d) Explicar por qué la corrección en un modelo secuencialmente consistente no autoriza a usar sin más variables ordinarias en un programa real con memoria débil.

58. Instrucción atómica `swap`. Se dispone de `swap(x,y)`, que intercambia atómicamente los valores booleanos de `x` e `y`. Diseñar protocolos de entrada y salida para $N \geq 2$ procesos mediante una variable compartida `ocupado`, inicialmente falsa, y una variable booleana local por proceso. Demostrar la exclusión mutua y discutir el progreso, la inanición y el coste de espera ocupada. ¿Basta la atomicidad de `swap` para asegurar equidad? Justificar.

59. Exclusión de capacidad k. Un recurso admite simultáneamente a lo sumo k procesos, con $1 \leq k < N$. Diseñar las subrutinas `entrar()` y `salir()` para N procesos utilizando únicamente una variable entera compartida `dentro`, inicialmente cero, y un cerrojo atómico `lock()` / `unlock()`. Se permite espera ocupada fuera del cerrojo. (a) Escribir pseudocódigo. (b) Demostrar el invariante $0 \leq$ `dentro` $\leq k$. (c) Explicar por qué no debe mantenerse el cerrojo mientras se espera a que haya capacidad. (d) Discutir si la solución asegura ausencia de inanición.

60. Algoritmo de turnos circulares. Para $N \geq 2$ procesos se comparte `s[0..N-1]`, inicialmente a cero. El proceso P_i espera hasta que se cumpla `s[i] != s[i-1]` si $i > 0$, o `s[0] = s[N-1]` si $i = 0$. Al salir, ejecuta `s[i]:=s[i-1]` si $i > 0$, o `s[0]:=1-s[0]` si $i = 0$. (a) Examinar la exclusión mutua. (b) ¿Qué sucede si un proceso permanece indefinidamente en su sección no crítica? (c) Determinar si se cumple el requisito de progreso cuando solamente un proceso desea entrar. (d) Distinguir entre alternancia forzosa y equidad entre procesos contendientes.

61. Tickets para dos procesos. Se comparten `n[0]=n[1]=0`. Cada `P(i)`, con `j=1-i`, ejecuta:

```
n[i] := 1;
n[i] := n[j] + 1;
while n[j] != 0 and
      (n[j] < n[i] or (n[j] = n[i] and j < i)) do
    skip;
endwhile;
<seccion critica>;
n[i] := 0;
```

(a) Estudiar la exclusión mutua, considerando todas las intercalaciones relevantes de las asignaciones iniciales. (b) Analizar si el protocolo puede producir interbloqueo o inanición. (c) Estudiar qué ocurre al eliminar la asignación `n[i]:=1`. (d) Explicar la función del desempate por identificador. No suponer que la lectura de `n[j]` y la escritura posterior constituyen una operación indivisible.

62. Peterson para N procesos: filtro por niveles. Considerar la generalización de Peterson con `nivel[i]` inicialmente cero y `victima[L]` compartidos. Cada proceso P_i recorre los niveles $L = 1, \ldots, N-1$:

```
for L := 1 to N-1 do
    nivel[i] := L;
    victima[L] := i;
    while exists j != i such that
          nivel[j] >= L and victima[L] = i do skip;
endfor;
<seccion critica>;
nivel[i] := 0;
```

(a) Justificar por qué no pueden coexistir dos procesos en sección crítica. (b) Analizar si pueden encontrarse dos procesos en el nivel $N-2$ mientras ninguno está en $N-3$; razonar según la

definición precisa de "estar en un nivel". (c) Examinar qué hipótesis adicionales serían necesarias para establecer cotas de espera. Evitar identificar automáticamente ausencia de inanición con espera acotada.

63. Algoritmo de Dijkstra para N procesos: modificación de una guarda. Se comparte `turno` y el vector `estado[i]` con valores `pasivo`, `solicitando` y `enSC`. En el algoritmo de Dijkstra estudiado en clase, durante el protocolo de entrada, un proceso que espera su turno ejecuta:

```
while turno != i do
    if estado[turno] = pasivo then turno := i;
endwhile;
```

Se propone sustituir la guarda por `estado[turno] != enSC`, sin alterar el resto del algoritmo. (a) Comparar las dos condiciones. (b) Construir, si es posible, una ejecución que invalide alguna propiedad de corrección. (c) Precisar si el defecto afecta a la seguridad, al progreso o a la equidad. Para el análisis, utilizar el protocolo completo de Dijkstra presentado en clase, no únicamente el fragmento anterior.